

# ML FRAUD DETECTION PIPELINE

## Key Terms, Terminologies & Parameters

A Complete Reference Guide Across All 8 Pipeline Stages

Nova Pay Fraud Detection System · Dr Woro Offeh · 2026

### Introduction

This reference guide explains every technical term, algorithm, metric, and parameter encountered across an end-to-end machine learning fraud detection pipeline. Each of the eight stages is documented with definitions written for a learner audience — precise enough for implementation, plain enough to understand without prior ML experience.

The guide is structured to support both study and reference. Read sequentially to build understanding from data collection through to model explanation, or jump to any stage using the pipeline overview below.

### The 8-Stage Pipeline at a Glance

|   |                                     |
|---|-------------------------------------|
| 1 | Data Collection & Understanding     |
| 2 | Data Cleaning & Feature Engineering |
| 3 | Handling Class Imbalance            |
| 4 | Model Training                      |
| 5 | Model Evaluation — Metrics          |
| 6 | Holdout Strategy & Validation       |
| 7 | Hyperparameter Tuning               |
| 8 | Model Interpretability — SHAP       |

### STEP 1 Data Collection & Understanding

The pipeline begins by gathering raw transaction data and developing a thorough understanding of its structure, quality, and the fraud problem you are trying to solve. Garbage in, garbage out — this step determines everything that follows.

| Term / Parameter | Meaning & Fraud Detection Impact                                                                                                                                                   |
|------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dataset          | The full collection of transactions used for training and evaluation. In fraud detection this typically contains millions of rows with a tiny fraction (0.1–2%) labelled as fraud. |

|                                 |                                                                                                                                                                                                                  |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Feature                         | A single input variable (column) used by the model. Examples: transaction amount, merchant category, time of day, device fingerprint, IP country.                                                                |
| Label / Target                  | The column indicating whether a transaction is fraud (1) or legitimate (0). Also called the dependent variable or ground truth.                                                                                  |
| Class imbalance                 | When one class (fraud) is far rarer than the other (legit). A 99:1 ratio is common. Naïve models learn to predict 'all legit' and still achieve 99% accuracy — <b>making recall useless without correction</b> . |
| EDA (Exploratory Data Analysis) | Statistical and visual inspection of the data before modelling. Reveals distributions, missing values, correlations, and early fraud signals.                                                                    |
| Fraud rate / Prevalence         | The percentage of transactions that are genuinely fraudulent. Low fraud rates (e.g. 0.5%) require special handling throughout the pipeline.                                                                      |
| Transaction log                 | The raw record of each financial event: timestamp, amount, merchant, card details, device info, location. The primary source of features.                                                                        |
| Ground truth                    | Confirmed fraud labels, usually validated by investigators or chargebacks. The quality of ground truth directly limits the maximum achievable model performance.                                                 |

**Key principle:**

Spend time here. Understand the fraud typology (card-not-present, account takeover, synthetic identity). Features you engineer in Step 2 must reflect real fraud mechanisms observed in this step.

## STEP 2 Data Cleaning & Feature Engineering

Raw transaction data is messy. This step transforms it into a clean, informative set of inputs that a model can learn from. Well-engineered features are often the single biggest lever for improving fraud detection.

| Term / Parameter             | Meaning & Fraud Detection Impact                                                                                                                                                            |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Missing value imputation     | Filling in absent data. Strategy matters: median imputation for amounts; 'Unknown' category for device type. Wrong imputation can mask fraud patterns (e.g. missing device = suspicious).   |
| Outlier handling             | Deciding what to do with extreme values. In fraud, outliers are often signals, not noise — a £50,000 transaction at 3am from a new device should not be capped away.                        |
| Encoding (one-hot / ordinal) | Converting categorical variables (e.g. merchant_category = 'electronics') into numbers a model can process. One-hot creates binary columns; ordinal preserves rank.                         |
| Normalisation / Scaling      | Rescaling numeric features to a common range (e.g. 0–1 or z-score). Critical for logistic regression; less important for tree models (XGBoost, LightGBM).                                   |
| Feature engineering          | Creating new variables from existing ones. Examples: velocity features (5 transactions in 10 minutes), deviation from baseline (amount 3× the user's average), time-since-last-transaction. |

|                     |                                                                                                                                                                                       |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Velocity features   | Count or sum of events in a rolling time window (e.g. transactions_last_1hr). Among the most powerful fraud signals — fraudsters act fast.                                            |
| Aggregated features | Summary statistics computed per user/card/merchant over a lookback window: mean amount, std of amounts, unique countries in 7 days.                                                   |
| Target encoding     | Replacing a category with the historical fraud rate for that category (e.g. merchant fraud rate). Powerful but leaks information if not done carefully within cross-validation folds. |
| Train/test leakage  | Accidentally using future or test-set information during training, causing unrealistically high metrics. Temporal leakage is the most common trap in fraud pipelines.                 |

#### Fraud insight:

Velocity and deviation features catch most card-not-present fraud. Engineer: (a) how fast the card is being used, (b) how different this transaction is from the cardholder's history, (c) whether the device/IP/email has appeared before.

### STEP 3 Handling Class Imbalance

Because fraud is rare, models default to predicting 'legitimate' for everything and still achieve high accuracy. This step forces the model to take fraud seriously.

| Term / Parameter                        | Meaning & Fraud Detection Impact                                                                                                                                                   |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SMOTE (Synthetic Minority Oversampling) | Creates artificial fraud examples by interpolating between existing fraud cases. Increases model exposure to fraud patterns. Apply only to training data, never to test data.      |
| Random oversampling                     | Duplicates existing fraud rows. Simpler than SMOTE. Risk: model memorises specific fraud transactions rather than learning general patterns.                                       |
| Random undersampling                    | Removes legitimate transactions to balance classes. Fast but discards real data. Use cautiously if training set is small.                                                          |
| class_weight = 'balanced'               | Instructs the model to penalise misclassifying fraud cases more heavily, in proportion to their rarity. The simplest and most robust fix for imbalance.                            |
| scale_pos_weight (XGBoost)              | Equivalent to class_weight for XGBoost. Set to (number of legit) / (number of fraud). E.g. 990/10 = 99. Tells XGBoost each fraud case counts as 99 legit cases.                    |
| is_unbalance (LightGBM)                 | LightGBM's built-in imbalance correction. Automatically reweights classes. Alternative: use scale_pos_weight as in XGBoost.                                                        |
| Threshold tuning                        | The model outputs a probability (0–1). The default decision boundary of 0.5 is rarely optimal for imbalanced fraud. Lowering to 0.2–0.3 increases recall at the cost of precision. |

**Common mistake:**

Never apply SMOTE before splitting into train/test sets. If you do, synthetic fraud examples derived from the test set contaminate training and inflate metrics by up to 15%. Always: split first, then oversample training only.

**STEP 4** Model Training

The cleaned, balanced dataset is used to fit a model — a mathematical function that learns the relationship between transaction features and fraud probability. Different algorithms make different assumptions and suit different data shapes.

| Term / Parameter                       | Meaning & Fraud Detection Impact                                                                                                                                                           |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Logistic Regression                    | A linear classifier. Draws one straight decision boundary. Fast, interpretable, and a useful baseline. Struggles with non-linear fraud patterns.                                           |
| C (regularisation)                     | Controls how flexible logistic regression's boundary is. Low C = simple rules, better generalisation. High C = fits training data closely, risk of memorising old fraud patterns.          |
| Random Forest                          | An ensemble of hundreds of decision trees, each trained on a random data subset. Majority vote decides. Captures non-linear patterns; robust to noisy features.                            |
| n_estimators                           | Number of trees in the forest / boosting rounds. More trees = more stable predictions. 100–500 is typical. Diminishing returns beyond 500.                                                 |
| max_depth                              | How many decision splits a tree can make. Deeper = more complex patterns learned. Too deep = model memorises training fraud, misses new fraud types. Typical: 3–8 for fraud.               |
| XGBoost                                | Gradient boosting: trees built sequentially, each one correcting the previous tree's mistakes. Highly accurate, handles missing values natively. Industry standard for tabular fraud data. |
| LightGBM                               | Same gradient boosting idea as XGBoost but uses histogram binning and leaf-wise tree growth. 3–10× faster, lower memory, handles very large transaction logs efficiently.                  |
| learning_rate (eta)                    | How much each new tree contributes to the final model. Smaller rate + more trees = more accurate but slower to train. Typical: 0.01–0.1.                                                   |
| subsample /<br>colsample_bytree        | Random fraction of rows/columns used per tree. Values 0.7–0.9 prevent overfitting and help the model generalise to unseen fraud patterns.                                                  |
| min_child_weight /<br>min_samples_leaf | Minimum number of transactions required in a terminal leaf node. Higher values = simpler trees = fewer false alarms from sparse fraud patterns.                                            |
| Gradient boosting                      | Iterative process: each new tree is fit to the residual errors (cases the model got wrong). Naturally focuses attention on hard-to-classify fraud cases.                                   |

### Recommendation for fraud:

Start with LightGBM as your primary model (speed + accuracy). Use logistic regression as an interpretable baseline. Ensemble both if production recall requirements are very high (>95%). Always use `class_weight / scale_pos_weight`.

## STEP 5 Model Evaluation — Metrics

Choosing the right metrics is as important as choosing the right model. Accuracy is misleading for fraud. These five metrics tell you what actually matters.

| Term / Parameter     | Meaning & Fraud Detection Impact                                                                                                                                                       |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Confusion matrix     | A 2×2 table showing True Positives (caught fraud), False Negatives (missed fraud), False Positives (false alarms), and True Negatives (correctly cleared legit).                       |
| True Positive (TP)   | A genuine fraud transaction that the model correctly flagged. The goal is to maximise these.                                                                                           |
| False Negative (FN)  | A genuine fraud transaction that the model missed. The costliest error in fraud detection — fraudsters get through.                                                                    |
| False Positive (FP)  | A legitimate transaction incorrectly flagged as fraud. Causes customer friction, declined cards, investigator workload.                                                                |
| Recall (Sensitivity) | $TP / (TP + FN)$ . Fraction of all actual fraud cases caught. The primary metric for fraud detection. Target: >85% for most fraud programmes.                                          |
| Precision            | $TP / (TP + FP)$ . Fraction of fraud alerts that are genuine fraud. Low precision = too many false alarms. Balanced against recall via threshold tuning.                               |
| F1 Score             | Harmonic mean of precision and recall: $2 \times (P \times R) / (P + R)$ . The main single-number metric when both catching fraud and avoiding false alarms matter.                    |
| ROC-AUC              | Area under the Recall vs False-Alarm-Rate curve at all thresholds. Measures overall model discrimination. 0.5 = random; 1.0 = perfect. Useful for comparing models.                    |
| PR-AUC               | Area under the Precision vs Recall curve. More informative than ROC-AUC when fraud prevalence is very low (<5%). Directly reflects the cost-of-false-alarm vs missed-fraud tradeoff.   |
| Accuracy             | Overall fraction of correct predictions. Misleading for fraud: a model predicting 'all legit' on a 1% fraud dataset achieves 99% accuracy with 0% recall. Never use as primary metric. |
| Decision threshold   | The probability cut-off above which a transaction is flagged. Default 0.5. Lowering to 0.2–0.3 increases recall (catch more fraud) at the cost of more false alarms.                   |

**Rule of thumb:**

For executive reporting, lead with F1 and Recall. For model selection, use PR-AUC if fraud rate < 5%. Monitor ROC-AUC for general model health. Always report alongside the decision threshold used.

**STEP 6 Holdout Strategy & Validation**

How you split and evaluate data determines whether your reported metrics reflect real-world performance. A wrong split strategy can make a poor model look excellent.

| Term / Parameter              | Meaning & Fraud Detection Impact                                                                                                                                                                                        |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Holdout split                 | Dividing data into train, validation, and test sets before any modelling. The test set is touched only once, at the very end, to produce honest final metrics.                                                          |
| Training set (~70%)           | The data the model learns from. All feature engineering, SMOTE, and parameter tuning uses this set only.                                                                                                                |
| Validation set (~15%)         | Used to tune hyperparameters and decision thresholds during development. Seen multiple times but never used to fit model weights directly.                                                                              |
| Test set (~15%)               | The final, untouched holdout. Seen exactly once. Provides an unbiased estimate of how the model performs on future transactions.                                                                                        |
| Temporal split                | Splitting by time rather than randomly. Train on months 1–10, validate on month 11, test on month 12. Critical for fraud: random splits allow future patterns to leak into training.                                    |
| Cross-validation (CV)         | Training and evaluating the model on multiple different splits of the training data to get a stable performance estimate. Standard: 5-fold or time-series CV.                                                           |
| Time-series CV (walk-forward) | A rolling validation strategy: train on months 1–3, test on month 4; train on 1–4, test on 5; etc. Mimics real deployment where the model sees only past data.                                                          |
| Data leakage                  | Any situation where information from the test set (or future data) influences model training. Leakage causes inflated metrics that collapse in production.                                                              |
| Overfitting                   | Model performs well on training data but poorly on new data. Happens when a model memorises training fraud patterns rather than learning general ones. Detected by a large gap between training and validation metrics. |
| Underfitting                  | Model is too simple to capture fraud patterns. High false negatives even on training data. Fix: more complex model or richer features.                                                                                  |

**Critical rule:**

Always split by time for fraud. Never use random splits. A fraudster's behaviour in December is different from March — a random split leaks December patterns into the March training set and produces optimistic recall figures that will not hold in production.

## STEP 7 Hyperparameter Tuning

Hyperparameters are settings you choose before training — the model does not learn them from data. Tuning them systematically finds the combination that maximises fraud detection performance on your specific dataset.

| Term / Parameter                                         | Meaning & Fraud Detection Impact                                                                                                                                                         |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hyperparameter                                           | A model configuration value set before training (e.g. number of trees, learning rate, max depth). Different from model parameters, which are learned from data (e.g. tree split values). |
| Grid search                                              | Exhaustive search over all specified hyperparameter combinations. Thorough but slow. Practical for small search spaces (2–3 parameters, few values each).                                |
| Random search                                            | Samples random combinations from the hyperparameter space. Finds good configurations faster than grid search for large spaces. Recommended as a first pass.                              |
| Bayesian optimisation                                    | Uses past trial results to intelligently select the next combination to try. Most efficient for expensive models. Tools: Optuna, Hyperopt, scikit-optimize.                              |
| Optuna / Hyperopt                                        | Python libraries for automated hyperparameter optimisation. Optuna is widely used for XGBoost and LightGBM tuning in fraud pipelines.                                                    |
| Scoring metric for tuning                                | The metric optimised during search. For fraud, use F1 or PR-AUC, not accuracy. Specify: <code>GridSearchCV(scoring='f1')</code> or <code>scoring='roc_auc'</code> .                      |
| Early stopping                                           | Halts training when validation performance stops improving. Prevents overfitting to old fraud patterns and reduces training time. Used with XGBoost and LightGBM.                        |
| <code>num_boost_round</code> / <code>n_estimators</code> | Maximum number of boosting trees. With early stopping, the optimal value is found automatically.                                                                                         |
| Learning rate schedule                                   | Gradually reducing the learning rate during training. Allows fine-grained adjustments later in training without overshooting. Common: step decay or cosine annealing.                    |

### Practical approach:

For fraud: (1) Run random search over `learning_rate`, `max_depth`, `n_estimators`, subsample with 50 trials; (2) Use F1 or PR-AUC as scoring; (3) Apply early stopping; (4) Refine around the best region with a narrower grid. Total time: <2 hours on a typical fraud dataset.

## STEP 8 Model Interpretability — SHAP

A model that cannot explain its decisions is difficult to trust, deploy, and regulate. SHAP (SHapley Additive exPlanations) translates any model's output into per-feature contributions for every individual prediction.

| Term / Parameter                     | Meaning & Fraud Detection Impact                                                                                                                                                          |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SHAP (SHapley Additive exPlanations) | A method rooted in cooperative game theory. Assigns each feature a contribution score (positive = pushes toward fraud, negative = pushes toward legit) for every individual prediction.   |
| SHAP value                           | The marginal contribution of one feature to the model's output for one transaction, relative to the average prediction. Units match the model's output (log-odds or probability).         |
| Base value                           | The model's average prediction across all training transactions. Each SHAP value adjusts up or down from this baseline.                                                                   |
| Force plot                           | A visual for one transaction showing each feature's SHAP value as arrows pushing the score toward or away from 'fraud'. Useful for explaining individual case decisions to investigators. |
| Summary plot (beeswarm)              | A chart showing SHAP values for all features across many transactions. Reveals which features matter most globally and how they interact with fraud probability.                          |
| SHAP waterfall plot                  | Ordered bar chart showing how each feature cumulatively builds up a single fraud prediction from the base value to the final score.                                                       |
| Feature importance (SHAP)            | Ranking features by their mean absolute SHAP value across all transactions. More reliable than built-in tree importance, which can mislead on correlated features.                        |
| SHAP interaction values              | Measures how two features jointly affect predictions. Example: high amount AND new device together may have a synergistic fraud signal beyond their individual effects.                   |
| Model drift detection via SHAP       | Monitoring which features dominate SHAP values over time. If 'new_device' suddenly becomes the top predictor, fraudsters may have changed tactics and retraining is needed.               |
| Regulatory explainability            | Many regulators (e.g. FCA, PRA) and internal audit functions require evidence that automated fraud decisions can be explained to affected customers. SHAP provides the audit trail.       |
| TreeExplainer                        | The SHAP algorithm optimised for tree-based models (Random Forest, XGBoost, LightGBM). Fast and exact. Usage: <code>shap.TreeExplainer(model).shap_values(X_test)</code> .                |
| LIME (alternative)                   | Local Interpretable Model-agnostic Explanations. An alternative to SHAP that builds a simple approximation around each prediction. Less consistent than SHAP but works on any model type. |

Deployment use case:



In production fraud systems, SHAP explains why each transaction was flagged: 'Flagged because: amount £3,200 (unusually high for this card), IP country = Nigeria (cardholder is UK-based), device not seen before.' This drives faster investigator decisions and satisfies GDPR right-to-explanation requirements.

## Appendix — Parameter Quick Reference

Recommended starting-point values for a fraud detection model on an imbalanced dataset (0.5–2% fraud rate):

| Parameter                  | Model               | Recommended value | Effect on fraud detection                  |
|----------------------------|---------------------|-------------------|--------------------------------------------|
| <b>class_weight</b>        | Logistic Reg / RF   | 'balanced'        | Corrects for imbalance; boosts recall      |
| <b>C</b>                   | Logistic Reg        | 0.1 – 1.0         | Lower = simpler rules, less overfitting    |
| <b>n_estimators</b>        | RF / XGBoost / LGBM | 200 – 500         | More trees = stable predictions            |
| <b>max_depth</b>           | XGBoost / LGBM      | 4 – 6             | Controls complexity; avoid overfitting     |
| <b>learning_rate</b>       | XGBoost / LGBM      | 0.01 – 0.05       | Lower + more trees = better generalisation |
| <b>scale_pos_weight</b>    | XGBoost             | legit/fraud count | Essential imbalance correction             |
| <b>is_unbalance</b>        | LightGBM            | True              | Built-in imbalance correction              |
| <b>subsample</b>           | XGBoost / LGBM      | 0.8               | Reduces overfitting to old fraud patterns  |
| <b>colsample_bytree</b>    | XGBoost / LGBM      | 0.8               | Feature randomisation; generalisation      |
| <b>min_child_weight</b>    | XGBoost / LGBM      | 5 – 20            | Prevents splits on tiny fraud subsets      |
| <b>Decision threshold</b>  | All models          | 0.2 – 0.35        | Lower = more fraud caught, more FPs        |
| <b>Primary eval metric</b> | All models          | F1 / PR-AUC       | Appropriate for imbalanced fraud data      |

*Document prepared for the Nova Pay ML Fraud Detection project — Dr Ogheneworo Offeh, 2026. All parameter recommendations assume a supervised binary classification task on tabular transaction data.*